

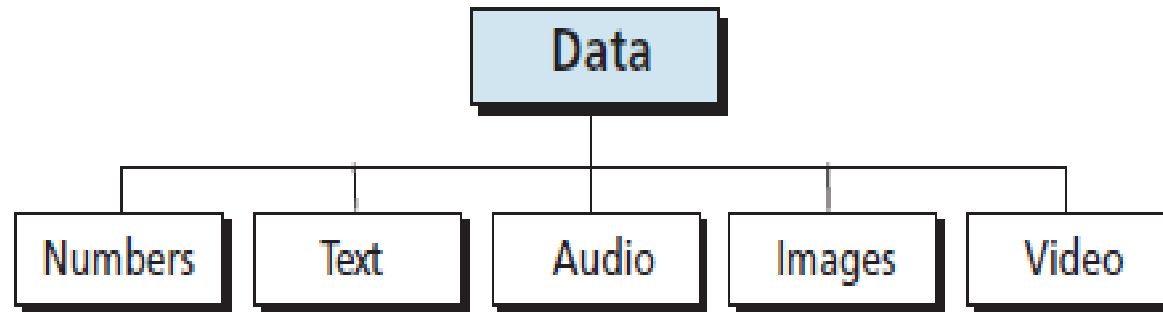
CHAPTER 3 Data Storage

- List five different data types used in a computer.
- Describe how different data is stored inside a computer.
- Describe how integers are stored in a computer.
- Describe how reals are stored in a computer.
- Describe how text is stored in a computer using one of the various encoding systems.
- Describe how audio is stored in a computer using sampling, quantization, and encoding.
- Describe how images are stored in a computer using raster and vector graphics schemes.
- Describe how video is stored in a computer as a representation of images changing in time.

3.1 INTRODUCTION

- Data today come in different forms including numbers, text, audio, image, and video (Figure 3.1).

Figure 3.1 *Different types of data*



The computer industry uses the term “**multimedia**” to define information that contains numbers, text, images, audio, and video.

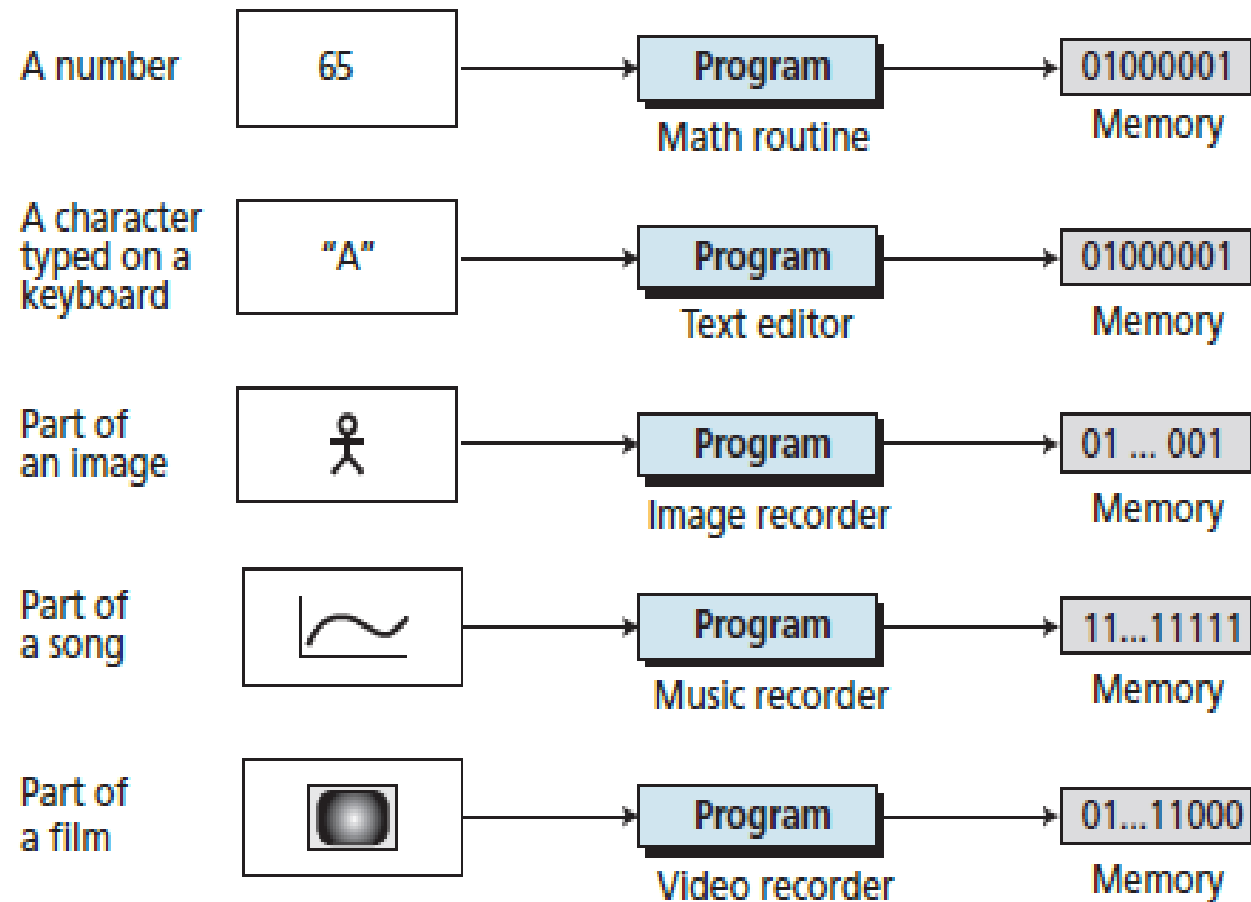
3.1.1 Data inside the computer

- All data types are transformed into a **uniform representation** when they are stored in a computer and transformed back to their original form when retrieved.
- This universal representation is called a **bit pattern**.

Figure 3.2 *A bit pattern*

1 0 0 0 1 0 1 0 1 1 1 1 1 0 1

Figure 3.3 *Storage of different data types*



3.1.2 Data compression

- To occupy less memory space, data is normally compressed before being stored in the computer.

3.1.3 Error detection and correction

- Another issue related to data is the detection and correction of errors during transmission or storage.

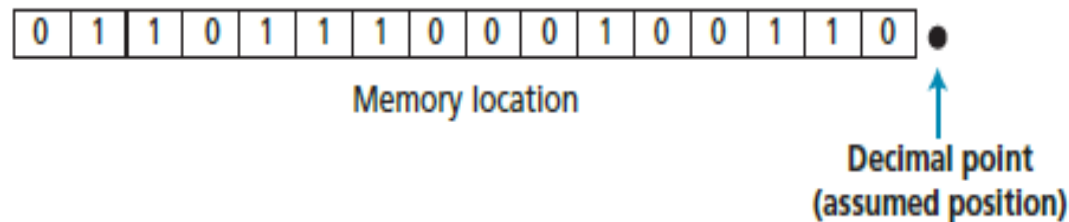
3.2 STORING NUMBERS

- A **number** is changed to the binary system before being stored in the computer memory, as described in Chapter 2.
- There are still two issues that need to be handled:
 - How to store **the sign** of the number.
 - How to show the **decimal point**.

3.2.1 Storing integers

- Integers are whole numbers (numbers without a fractional part).
 - For example, 134 and -125 are integers, whereas 134.23 and -0.235 are not.
- An integer can be thought of as a number in which the position of **the decimal point is fixed**: the decimal point is to the right of the least significant (rightmost) bit.
 - For this reason, fixed-point representation is used to store an integer, as shown in Figure 3.4.
- In this representation the decimal point is assumed but not stored.

Figure 3.4 *Fixed-point representation of integers*



An integer is normally stored in memory using fixed-point representation.

Unsigned representation 非負整數

- An **unsigned integer** is an integer that can never be negative and can take only 0 or positive values.
- Its range is between 0 and positive infinity.
- An input device stores an unsigned integer using the following steps:
 1. The integer is changed to binary.
 2. If the number of bits is less than n , 0s are added to the left.

Example 3.1 Store 7 in an 8-bit memory location using unsigned representation.

Change 7 to binary → 1 1 1

Add five bits at the left → 0 0 0 0 0 1 1 1

Solution:
(00000111)₂

Example 3.2 Store 258 in a 16-bit memory location.

Change 258 to binary $\rightarrow$ 1 0 0 0 0 0 0 1 0

Add seven bits at the left $\rightarrow$ 0 0 0 0 0 0 0 1 0 0 0 0 0 0 1 0

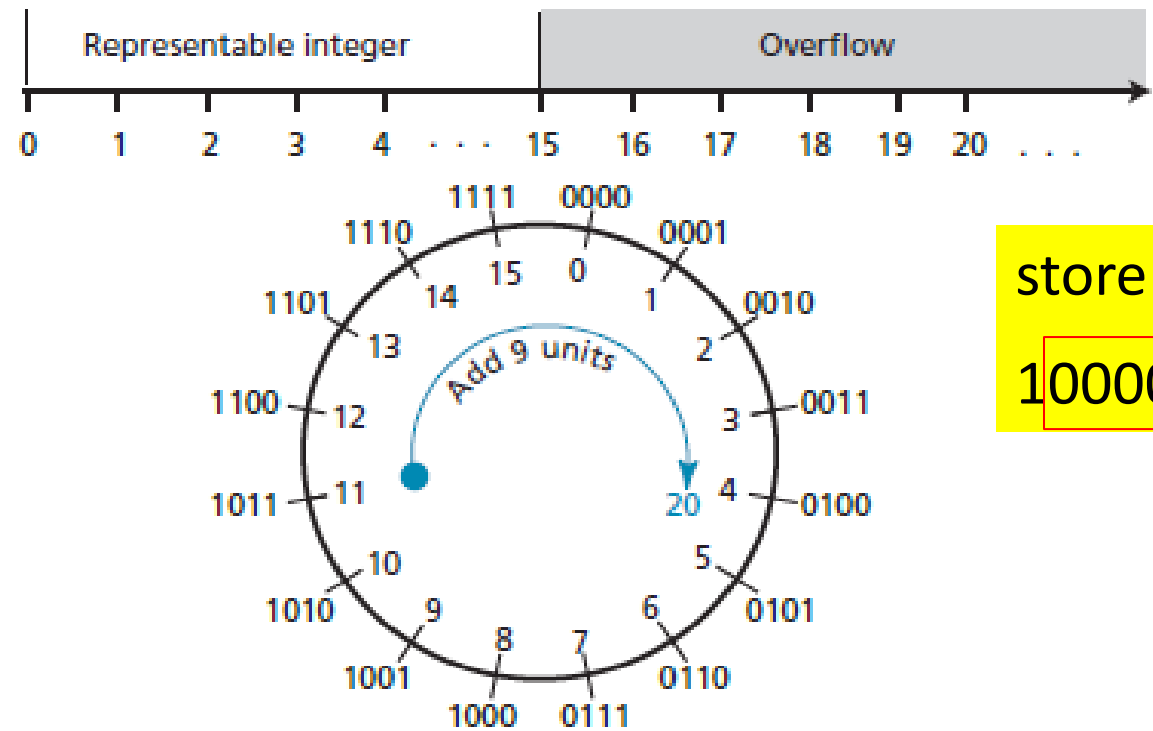
Ans: $(0000000100000010)_2$

Example 3.3 What is returned from an output device when it retrieves the bit string 00101011 stored in memory as an unsigned integer.

Solution: $00101011 = 32 + 8 + 2 + 1 = 43$

Figure 3.5 shows what happens if we try to store an integer that is larger than $2^4 - 1 = 15$ in a memory location that can only hold 4 bits.

Figure 3.5 *Overflow in unsigned integers*



store 16
10000

Sign-and-magnitude representation 符號與數值表示法 (有號整數)

- In this method, the available range for unsigned integers (0 to $2^n - 1$) is divided into two equal sub-ranges.
- The first half represents positive integers, the second half, negative integers.

Figure 3.6 Sign-and-magnitude representation

0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
0	1	2	3	4	5	6	7	-0	-1	-2	-3	-4	-5	-6	-7

- In sign-and-magnitude representation, **the leftmost bit** defines the sign of the integer.
 - If it is **0**, the integer is **positive**.
 - If it is **1**, the integer is **negative**.

Example 3.4 Store **+28** in an **8-bit** memory location using sign-and-magnitude representation.

Change 28 to 7-bit binary

0 0 1 1 1 0 0

Add the sign and store

0

0 0 1 1 1 0 0

Example 3.5 Store **−28** in an **8-bit** memory location using sign-and-magnitude representation.

Change 28 to 7-bit binary

0 0 1 1 1 0 0

Add the sign and store

1

0 0 1 1 1 0 0

Example 3.6 Retrieve the integer that is stored as 01001101 in sign-and-magnitude representation.

Solution

Leftmost bit is 0, the sign is +

The rest of the bits (1001101) are changed to decimal: $64 + 8 + 4 + 1 = 77$

+77

Example 3.7 Retrieve the integer that is stored as 10100001 in sign-and-magnitude representation.

Solution

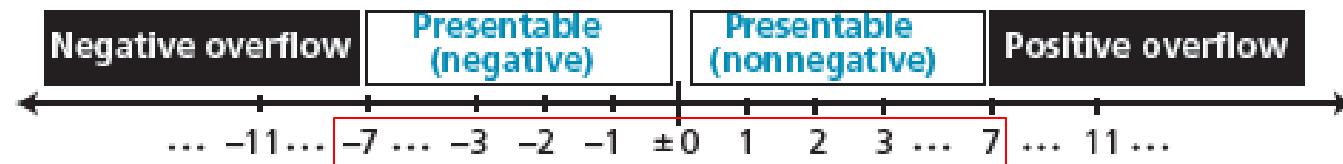
Leftmost bit is 1, the sign is -

The rest of the bits (0100001) are changed to decimal: $32 + 1 = 33$

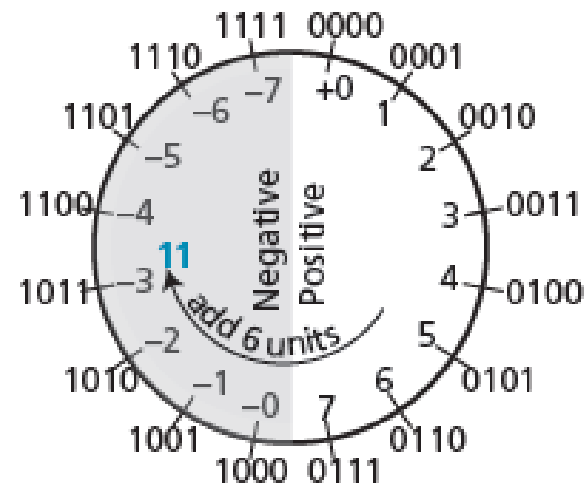
-33 (課本p.50的-17錯誤)

Figure 3.7 shows both positive and negative overflow when storing an integer in sign-and-magnitude representation using a 4-bit memory location.

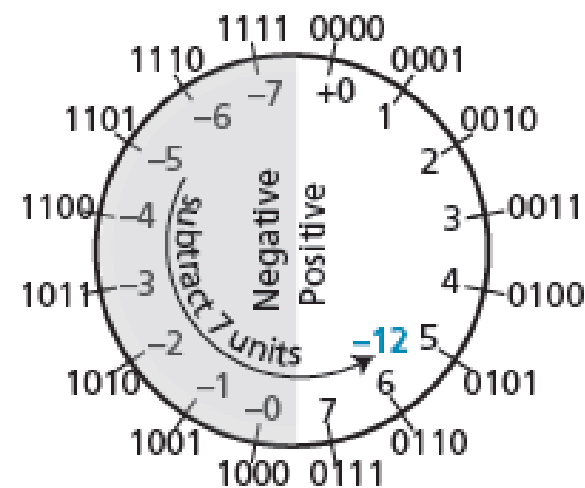
Figure 3.7 *Overflow in sign-and-magnitude representation*



a. Linear characteristic of an integer in sign-and-magnitude format



b. Positive overflow

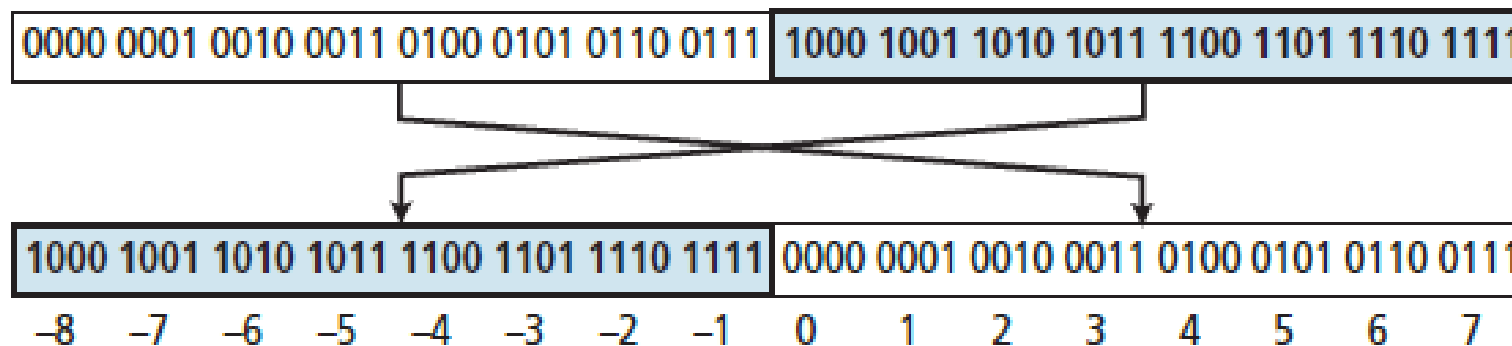


c. Negative overflow

Two's complement representation 二補數表示法

- Almost all computers use two's complement representation to store a signed integer in an n -bit memory location.
- In this method, the available range for an unsigned integer of $(0 \text{ to } 2^n - 1)$ is divided into two equal sub-ranges. The first sub-range is used to represent nonnegative integers, the second half to represent negative integers.

Figure 3.8 Two's complement representation



The leftmost bit defines the sign of the integer.

0 the integer is positive.

1 the integer is negative.

One's Complementing 一補數 – taking the one's complement of an integer

- The operation can be applied to any integer, positive or negative.
- This operation simply reverses (flips) each bit. A 0-bit is changed to a 1-bit, and a 1-bit is changed to a 0-bit

Example 3.8 Take the **one's complement** of the integer 00110110.

Original pattern	0	0	1	1	0	1	1	0
After applying one's complement operation	1	1	0	0	1	0	0	1

Example 3.9 Get the original integer applying the **one's complement** operations **twice**.

Original pattern	0	0	1	1	0	1	1	0
One's complementing once	1	1	0	0	1	0	0	1
One's complementing twice	0	0	1	1	0	1	1	0

Two's Complementing - taking the two's complement of an integer in binary

- This operation is done in two steps.
 1. copy bits from the right until a 1 is copied
 2. flip the rest of the bits

Example 3.10 The following shows how we take the two's complement of the integer 00110100.

Original integer	0	0	1	1	0	1	0	0
	↓	↓	↓	↓	↓	↓	↓	↓
Two's complementing once	1	1	0	0	1	1	0	0

Example 3.11 Get the original integer if we apply the **two's complement** operation twice.

Original integer	0	0	1	1	0	1	0	0
	↓	↓	↓	↓	↓	↓	↓	↓
Two's complementing once	1	1	0	0	1	1	0	0
	↓	↓	↓	↓	↓	↓	↓	↓
Two's complementing twice	0	0	1	1	0	1	0	0

Example 3.12 Store the integer 28 in an 8-bit memory location using two's complement representation.

Solution

Change 28 to 8-bit binary

0 0 0 1 1 1 0 0

- $28 = 16 + 8 + 4 = 2^4 + 2^3 + 2^2$
- Note that five extra 0s are added to the left of the integer to make it eight bits.

Example 3.13 Store -28 in an 8-bit memory location using two's complement representation.

Solution

Change 28 to 8-bit binary	0	0	0	1	1	1	0	0
	↓	↓	↓	↓	↓	↓	↓	↓
Apply two's complement operation	1	1	1	0	0	1	0	0

Example 3.14 Retrieve the integer that is stored as 00001101 in memory in two's complement format.

Solution

Leftmost bit is 0. The sign is positive	0	0	0	0	1	1	0	1
Integer changed to decimal								13
Sign is added (optional)								+13

Example 3.15 Retrieve the integer that is stored as 11100110 in memory using two's complement format.

Solution

- The leftmost bit is 1, so the integer is negative.
- The integer needs to be two's complemented before changing to decimal.

Leftmost bit is 1. The sign is negative

1	1	1	0	0	1	1	0
↓	↓	↓	↓	↓	↓	↓	↓

Apply two's complement operation

0	0	0	1	1	0	1	0
---	---	---	---	---	---	---	---

Integer changed to decimal

26

Sign is added

-26

An alternative way to take the two's complement of an integer

take the one's complement then add 1 to the result

Original integer	1 0 0 0 1 1 0 1
------------------	-----------------

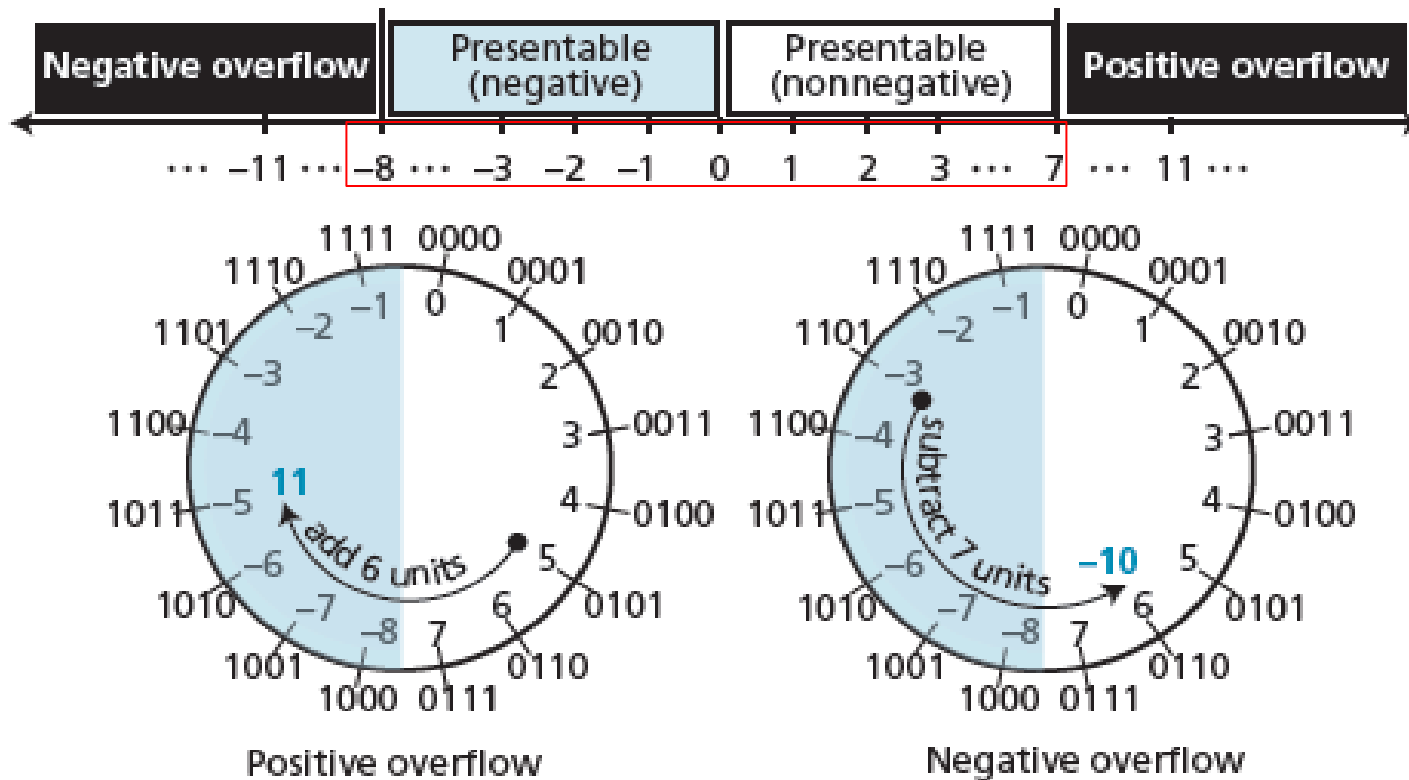
One's complementing	0 1 1 1 0 0 1 0
---------------------	-----------------

+	1
---	---

Two's complementing	0 1 1 1 0 0 1 1
---------------------	-----------------

There is only one zero in two's complement notation.

Figure 3.9 *Overflow in two's complement representation*



3.2.2 Comparison of the three systems

<i>Contents of memory</i>	<i>Unsigned</i>	<i>Sign-and-magnitude</i>	<i>Two's complement</i>
0000	0	0	+0
0001	1	1	+1
0010	2	2	+2
0011	3	3	+3
0100	4	4	+4
0101	5	5	+5
0110	6	6	+6
0111	7	7	+7
1000	8	-0	-8
1001	9	-1	-7
1010	10	-2	-6
1011	11	-3	-5
1100	12	-4	-4
1101	13	-5	-3
1110	14	-6	-2
1111	15	-7	-1

3.2.3 Storing reals

- **A real** is a number with an **integral part** and a **fractional part**.
- For example, 23.7 is a real number
 - the integral part is 23
 - the fractional part is $0.7 = 7/10$.
- Although a fixed-point representation can be used to represent a real number, the result may not be accurate or it may not have the required precision.
- The next two examples explain why.

Example 3.16 In the decimal system, assume that we use a fixed-point representation with 2 digits at the right of the decimal point and 14 digits at the left of the decimal point, for a total of 16 digits. The precision of a real number in this system is lost if we try to represent a decimal number such as 1.00234: the system stores the number as 1.00.

													1	•	0	0
--	--	--	--	--	--	--	--	--	--	--	--	--	---	---	---	---

Example 3.17 In the decimal system, assume that we use a fixed-point representation with 6 digits to the right of the decimal point and 10 digits for the left of the decimal point, for a total of sixteen digits. The accuracy of a real number in this system is lost if we try to represent a decimal number such as 236154302345.00. The system stores the number as 6154302345.00: the integral part is much smaller than it should be.

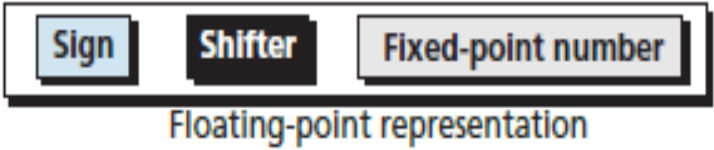
6	1	5	4	3	0	2	3	4	5	•	0	0				
---	---	---	---	---	---	---	---	---	---	---	---	---	--	--	--	--

Real numbers with very large integral parts or very small fractional parts should not be stored in fixed-point representation.

Floating-point representation 浮點數表示法

- The solution for maintaining accuracy or precision is to use **floating-point representation**.

Figure 3.10 The three parts of a real number in floating-point representation



A **floating point** representation of a number is made up of three parts: a **sign**, a **shifter**, and a **fixed-point number**.

Example 3.18 The following shows the decimal number 7,452,000,000,000,000,000.00 in scientific notation (floating-point representation).

Actual number	→	+	7,425,000,000,000,000,000.00	sign (+)
Scientific notation	→	+	7.425×10^{21}	shifter (21) the exponent
				fixed-point part (7.425)

Example 3.19 Show the number -0.00000000000000232 in scientific notation (floating-point representation).

Actual number	→	–	0.00000000000000232	sign (–) shifter (–14) fixed-point part (2.32)
Scientific notation	→	–	2.32×10^{-14}	

Example 3.20 Show the number $(10100100000000000000000000000000.00)_2$ in floating-point representation.

Actual number	→	+	$(10100100000000000000000000000000.00)_2$
Scientific notation	→	+	1.01001×2^{32}

Example 3.21 Show the number $-(0.000000000000000000000000101)_2$ in floating-point representation.

Actual number	→	–	$(0.000000000000000000000000101)_2$
Scientific notation	→	–	1.01×2^{-24}

Normalization

- To make the fixed part of the representation uniform, both the scientific method (for the decimal system) and the floating-point method (for the binary system) use **only one non-zero digit** on the left of the decimal point.
- In the decimal system this digit can be 1 to 9, while in the binary system it can only be 1. In the following, d is a non-zero digit, x is a digit, and y is either 0 or 1.

Decimal $\rightarrow$ $\pm$ $d.xxxxxxxxxxxxxx$

Note: d is 1 to 9 and each x is 0 to 9

Binary $\rightarrow$ $\pm$ $1.yyyyyyyyyyyyyyy$

Note: each y is 0 or 1

Sign, exponent, and mantissa

- After a binary number is normalized, only the three pieces of information about the number are stored: sign, exponent, and mantissa.
- For example, +1000111.0101 becomes

<i>Sign</i>	<i>Exponent</i>		<i>Mantissa</i>
+	2^6	$\times$	1.0001110101
0	6		0001110101

Textbook: p. 58

- Note that the point and **the bit 1 to the left of the fixed-point section are not stored**—they are implicit.
- The mantissa is a fractional part that, together with the sign, is treated like an integer stored in sign-and-magnitude representation.

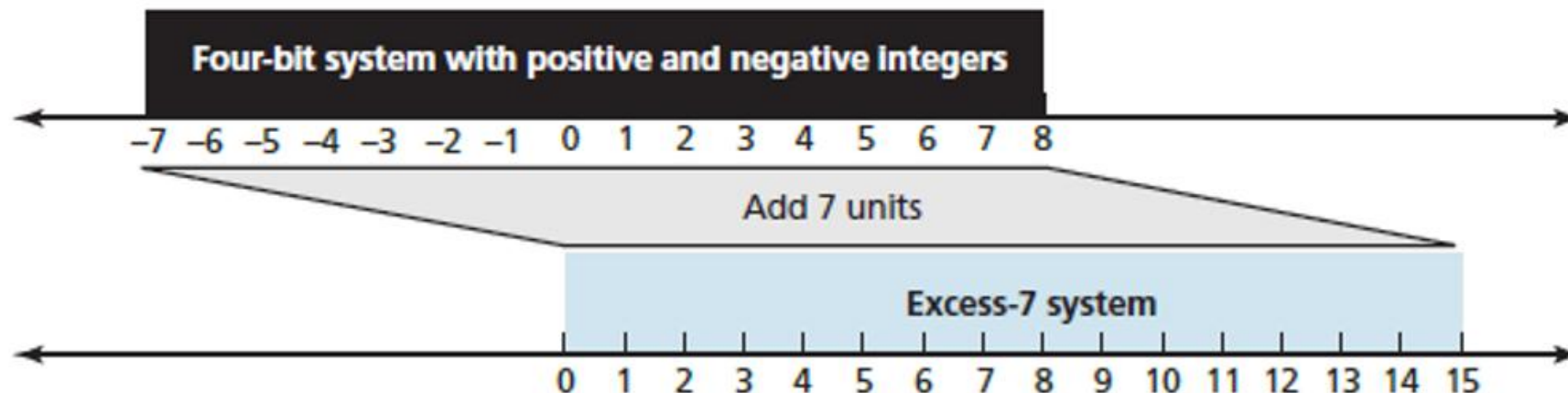
Excess System 偏移表示法

- The exponent, the power that shows how many bits the decimal point should be moved to the left or right, is a signed number. Although this could have been stored using two's complement representation, a new representation, called the Excess system, is used instead.
- In the Excess system, both **positive** and **negative** integers are stored as **unsigned integers**.
- To represent a positive or negative integer, a positive integer (called a bias) is added to each number to shift them uniformly to the non-negative side.
- The value of this bias is $2^{m-1} - 1$, where m is the size of the memory location to store the exponent.

Example 3.22 We can express sixteen integers in a number system with 4-bit allocation.

- By adding seven units to each integer in this range, we can uniformly translate all integers to the right and make all of them positive without changing the relative position of the integers with respect to each other, as shown in the figure.
- The new system is referred to as **Excess-7**, or biased representation with biasing value of 7.

Figure 3.11 *Shifting in Excess representation*

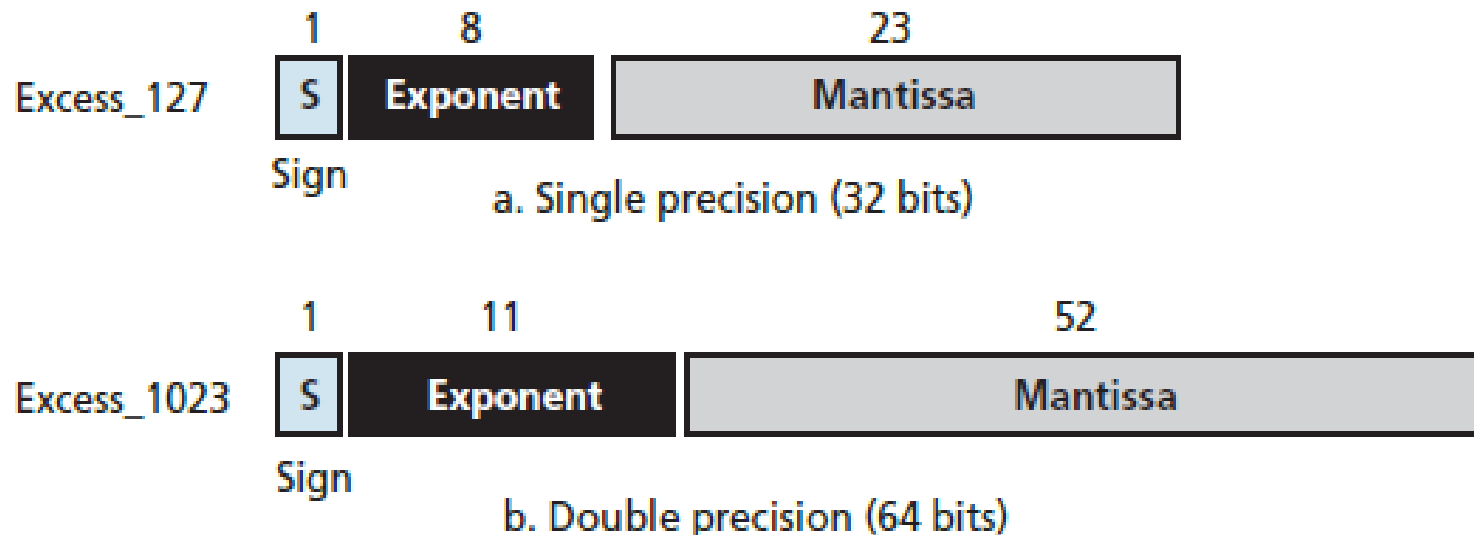


IEEE Standard (IEEE 754 Floating Point Numbers) Concept 概念

The Institute of Electrical and Electronics (IEEE) has defined several standards for storing float-point numbers.

- Two common representations: single precision and double precision.

Figure 3.12 IEEE standards for floating-point representation



IEEE Specifications

Table 3.2 Specifications of the two IEEE floating-point standards

<i>Parameter</i>	<i>Single Precision</i>	<i>Double Precision</i>
Memory location size (number of bits)	32	64
Sign size (number of bits)	1	1
Exponent size (number of bits)	8	11
Mantissa size (number of bits)	23	52
Bias (integer)	127	1023

Example 3.23 Show the Excess_127 (single precision) representation of the decimal number 5.75.

Solution

- The sign is **positive**, so $S = 0$.
- Decimal to binary transformation: $5.75 = (101.11)_2$.
- Normalization: $(101.11)_2 = (1.0111)_2 \times 2^2$.
- $E = 2 + 127 = 129 = (10000001)_2$, $M = 0111$. We need to add 19 zeros at the right of M to make it 23 bits.

The number is stored in the computer as

0	1	0	0	0	0	0	0	1	0	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
S	E								M																							

Example 3.24 Show the Excess_127 (single precision floating point) representation of the decimal number

Solution

- The sign is **negative**, so **S = 1**.
- Decimal to binary transformation: $161.875 = (10100001.111)_2$.
- Normalization: $(10100001.111)_2 = (1.0100001111)_2 \times 2^7$.
- E** = 7 + 127 = 134 = **(10000110)**₂ and **M** = **(0100001111)**₂.

The number is stored in the computer as

1	1	0	0	0	0	1	1	0	0	1	0	0	0	0	0	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0
S	E								M																								

Example 3.25 Show the Excess_127 (single precision) representation of the decimal number -0.0234375 .

Solution

- $S = 1$ (the number is **negative**).
- Decimal to binary transformation: $0.0234375 = (0.0000011)_2$.
- Normalization: $(0.0000011)_2 = (1.\textcolor{teal}{1})_2 \times 2^{-6}$.
- $E = -6 + 127 = 121 = (\textcolor{red}{01111001})_2$ and $M = (\textcolor{teal}{1})_2$.

The number is stored in the computer as

[illegible]

Example 3.26 The bit pattern $(11001010000000000111000100001111)_2$ is stored in Excess_127 format. Show the value in decimal.

Solution

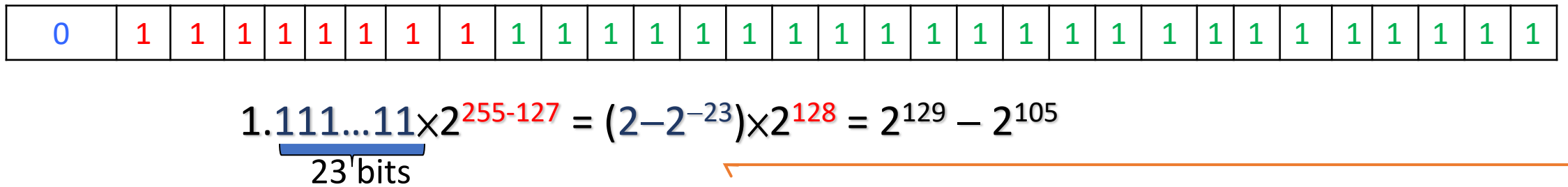
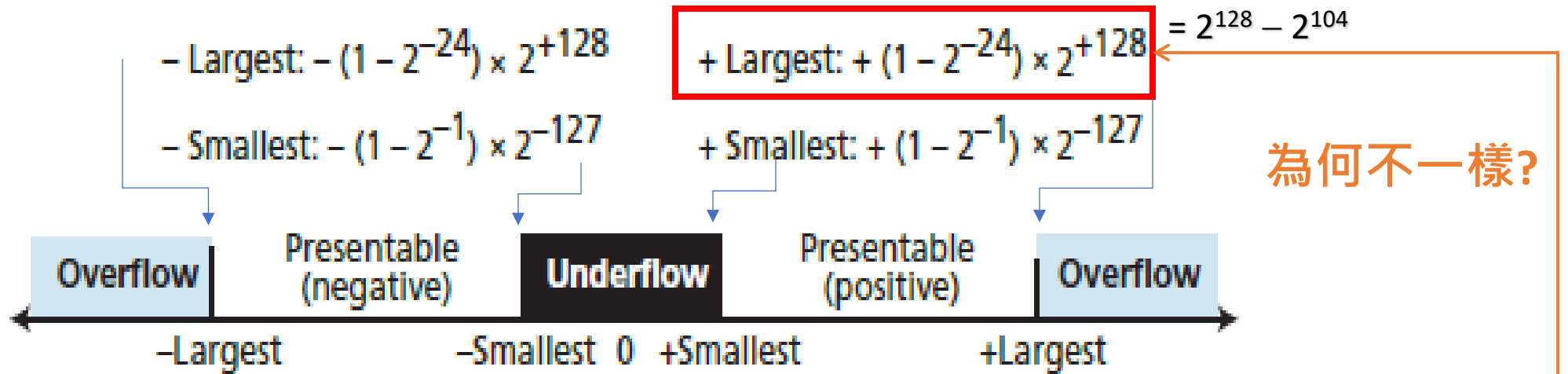
- a. The first bit represents S, the next eight bits, E and the remaining 23 bits, M.

S	E	M
1	10010100	00000000111000100001111

- b. The sign is negative.
c. The shifter = $E - 127 = 148 - 127 = 21$.
d. This gives us $(1.00000000111000100001111)_2 \times 2^{21}$.
e. The binary number is $(1000000001110001000011.11)_2$.
f. The absolute value is 2,104,378.75.
g. The number is **-2,104,378.75**.

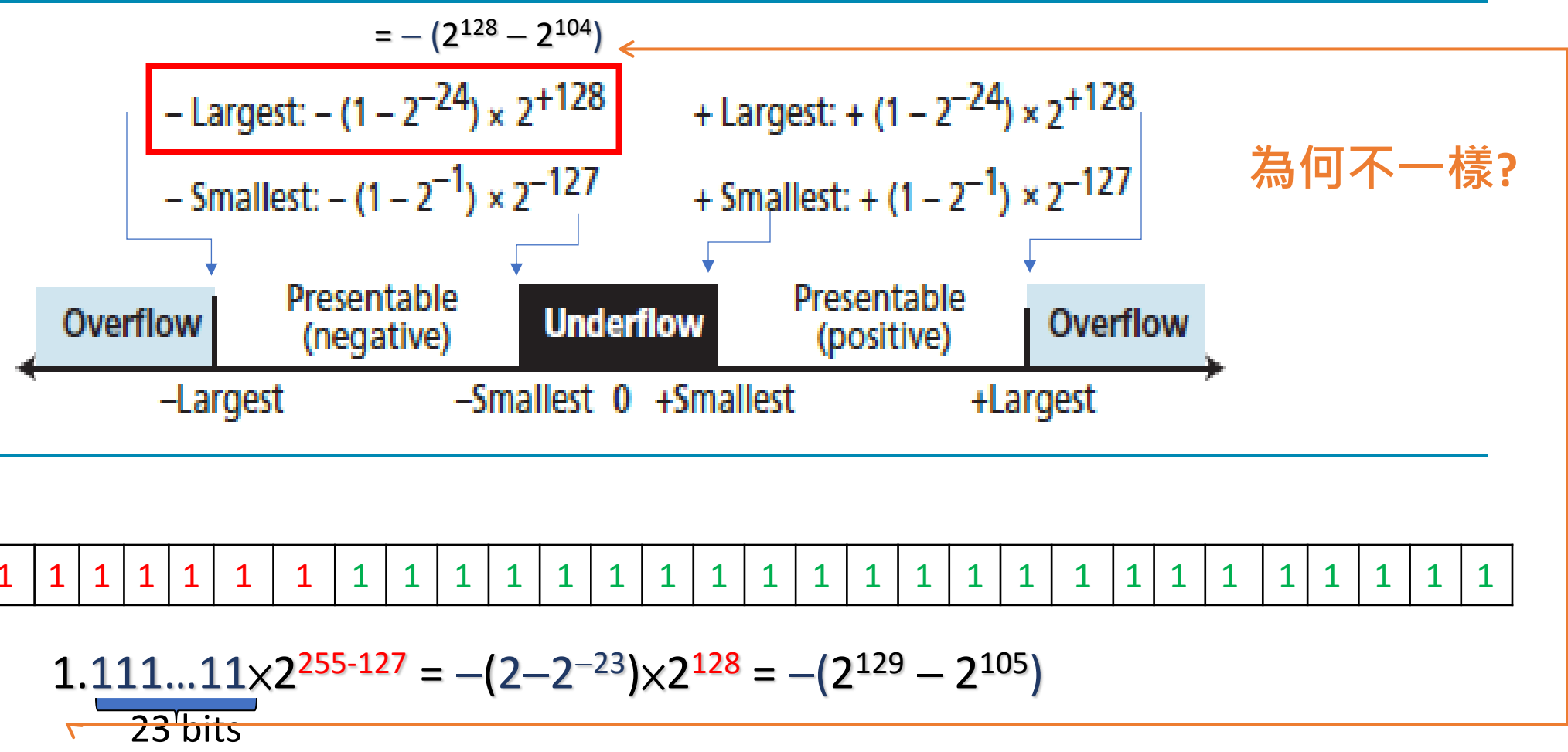
Overflow and Underflow

Figure 3.13 *Overflow and underflow in floating-point representation of reals*



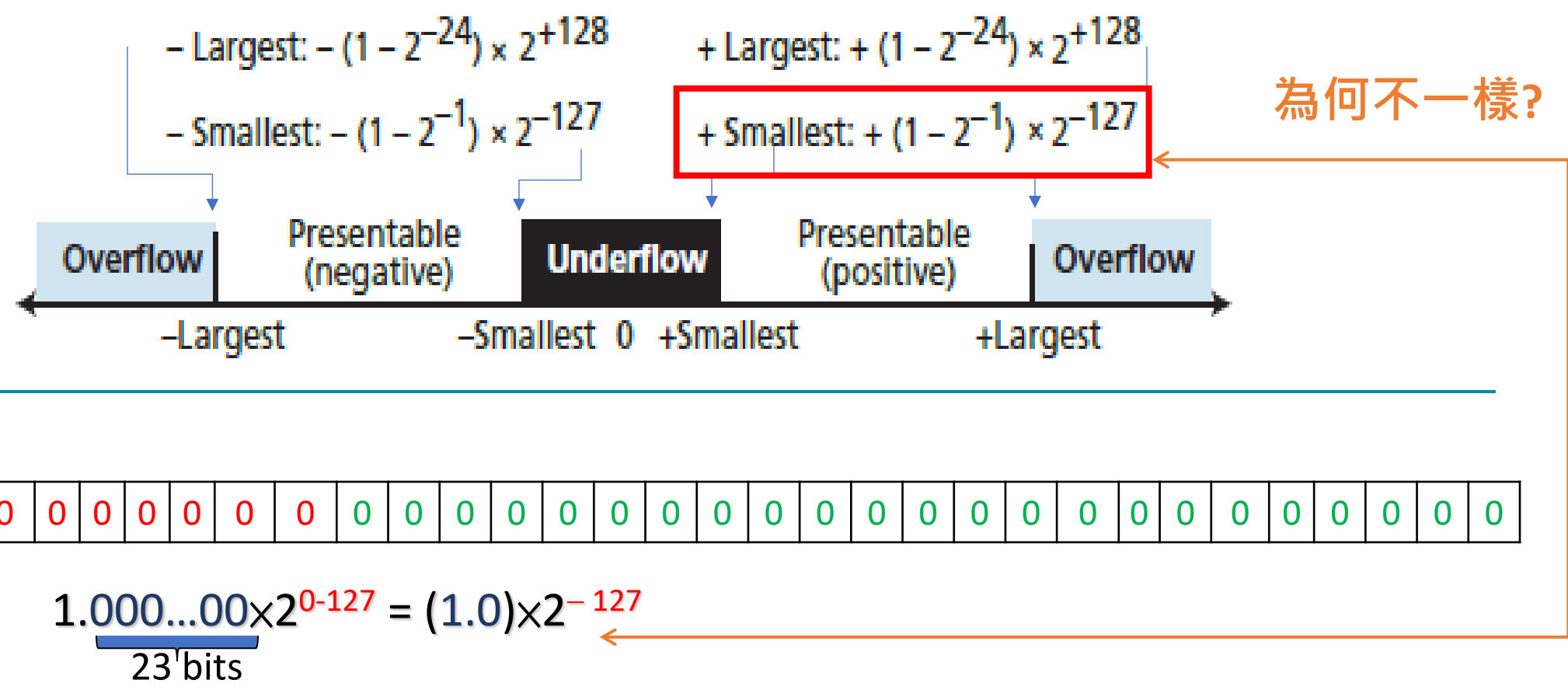
Overflow and Underflow

Figure 3.13 *Overflow and underflow in floating-point representation of reals*



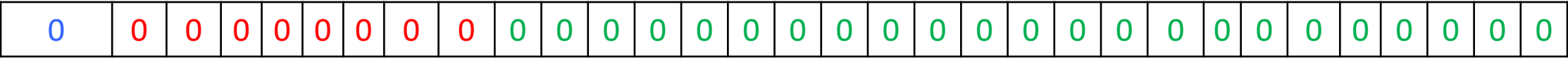
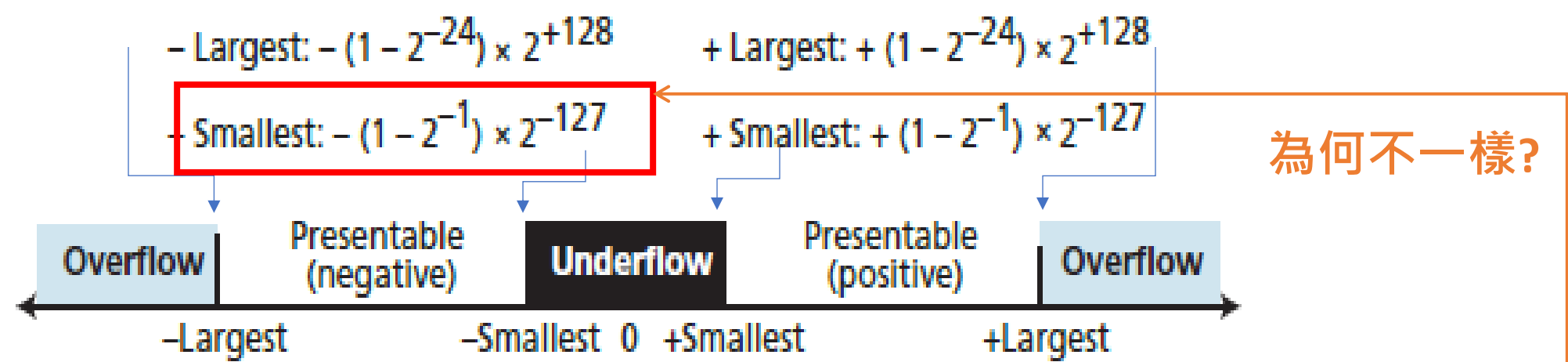
Overflow and Underflow

Figure 3.13 *Overflow and underflow in floating-point representation of reals*



Overflow and Underflow

Figure 3.13 *Overflow and underflow in floating-point representation of reals*



$$\underbrace{1.000\dots00}_{23 \text{ bits}} \times 2^{0-127} = -(1.0) \times 2^{-127}$$



Understanding IEEE 754 32-bit floating-point numbers in practice

sign exponent (8 bits)

fraction or mantissa (23 bits)



Sign (1 bit): 0 + 1 –

Exponent (8 bits): –126 to +127

–127(all 0s) and +128(all 1s) are reserved for special numbers

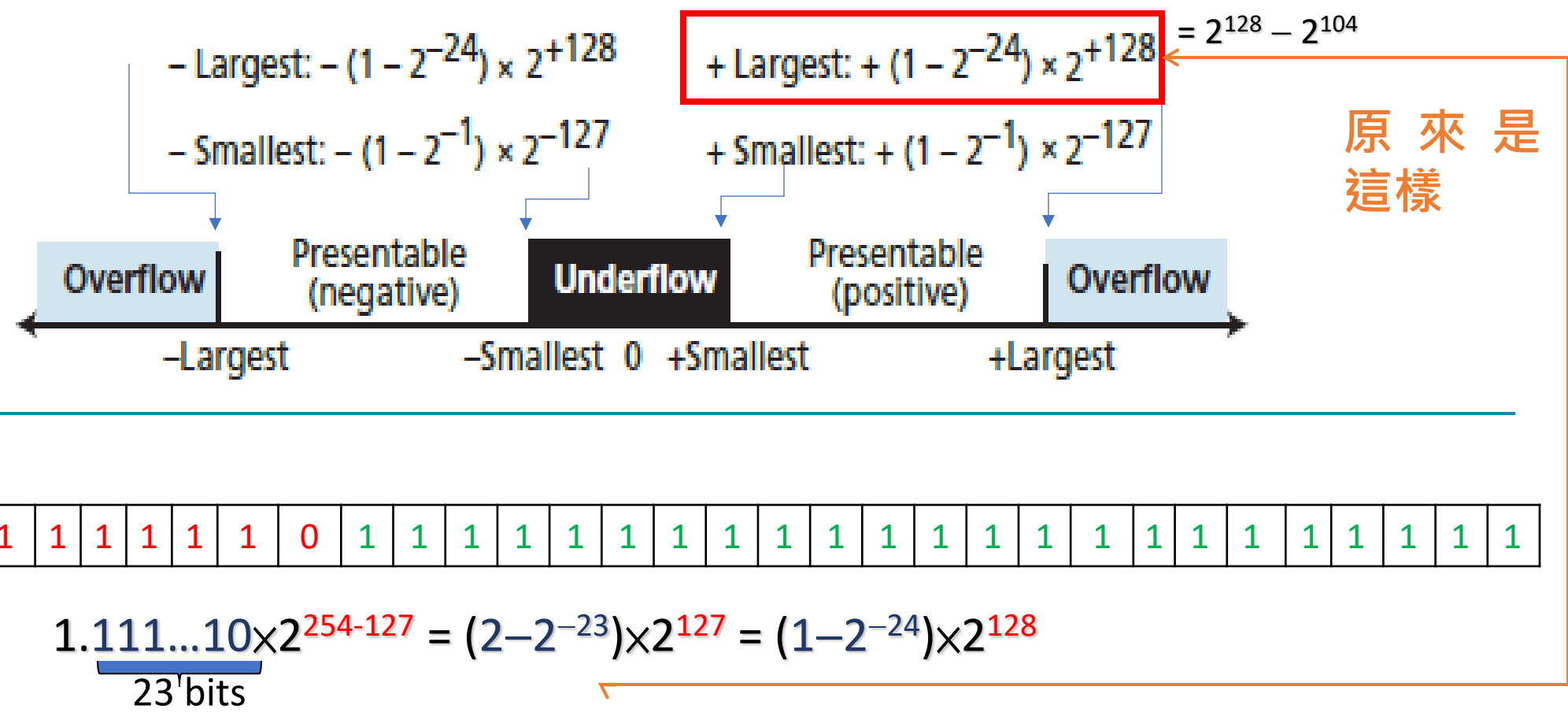
為何不一樣?
Exponent為-127和128有其他用途

Exponent	fraction=0	fraction≠0	Equation
00000000	± 0	subnormal number	$(-1)^{\text{sign}} \times 2^{-126} \times 0.\text{fraction}$
00000001 ... 11111110	normal value		$(-1)^{\text{sign}} \times 2^{\text{exponent}-127} \times 1.\text{fraction}$
11111111	$\pm \infty$	NaN(quiet, signaling)	fraction 最高位 = 1 → QNaN (Quiet NaN) fraction 最高位 = 0 → SNaN (Signaling NaN)

Mantissa(23 bits to the right of the binary point)

Overflow and Underflow

Figure 3.13 *Overflow and underflow in floating-point representation of reals*



Storing Zero

- A real number with an integral part and the fractional part set to zero, that is, 0.0, cannot be stored using the steps discussed above.
- To handle this **special case**, it is agreed that in this case the sign, exponent, and the mantissa are **set to 0s**.

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

- Smallest positive subnormal number = $2^{-126} \times 2^{-23} = 2^{-149}$

0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	1
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

			L	GRS
62831852		1	11011111010111100111011	00
	0 10011000		11011111010111100111011	0
62831853		1	11011111010111100111011	01
	0 10011000		11011111010111100111011	-1
62831854		1	11011111010111100111011	10
	0 10011000		11011111010111100111100	2
62831855		1	11011111010111100111011	11
	0 10011000		11011111010111100111100	1
62831856		1	11011111010111100111100	00
	0 10011000		11011111010111100111100	0
62831857		1	11011111010111100111100	01
	0 10011000		11011111010111100111100	-1
62831858		1	11011111010111100111100	10
	0 10011000		11011111010111100111100	-2
62831859		1	11011111010111100111100	11
	0 10011000		11011111010111100111101	1
62831860		1	11011111010111100111101	00
	0 10011000		11011111010111100111101	0
62831861		1	11011111010111100111101	01
	0 10011000		11011111010111100111101	-1
62831862		1	11011111010111100111101	10
	0 10011000		11011111010111100111110	2
62831863		1	11011111010111100111101	11
	0 10011000		11011111010111100111110	1

Mantissa overflow: Round to nearest, ties to even

硬體常用判斷：**GRS** 三位

為了決定是否進位，硬體會保留被丟棄區段的三個摘要位：

G (guard)：LSB 後的第 1 位

R (round)：第 2 位

S (sticky)：其餘所有更低位的 OR

在 **ties-to-even** 模式下，「是否進位」可用簡潔公式表示：

•進位條件： $\text{increment} = G \ \& \ (R \mid S \mid L)$

其中 **L** 是保留區的 LSB。

•若 $G=0 \rightarrow$ 小於半 ULP $\rightarrow$ 不進位。

•若 $G=1$ 且 $R \mid S=1 \rightarrow$ 大於半 ULP $\rightarrow$ 進位。

•若 $G=1$ 且 $R=S=0 \rightarrow$ 正好一半，此時 $\text{increment} = L$ ：

• $L=0$ （目前為偶） $\rightarrow$ 不進位（保持偶）。

• $L=1$ （目前為奇） $\rightarrow$ 進位（讓結果變偶）。

3.3 STORING TEXT

A section of text in any language is a sequence of symbols used to represent an idea in that language.

- For example, the English language uses
 - 26 symbols (A, B, C,..., Z) to represent uppercase letters, 26 symbols (a, b, c, ..., z) to represent lowercase letters
 - 10 symbols (0, 1, 2, ..., 9) to represent numeric characters
 - some symbols (., ?, :, ; , ..., !) to represent punctuation
 - Other symbols such as blank, newline, and tab are used for text alignment and readability.

- We can represent each symbol with a bit pattern.
- In other words, text such as “CATS”, which is made up from four symbols, can be represented as four n-bit patterns, each pattern defining a single symbol (Figure 3.14).

Figure 3.14 *Representing symbols using bit patterns*

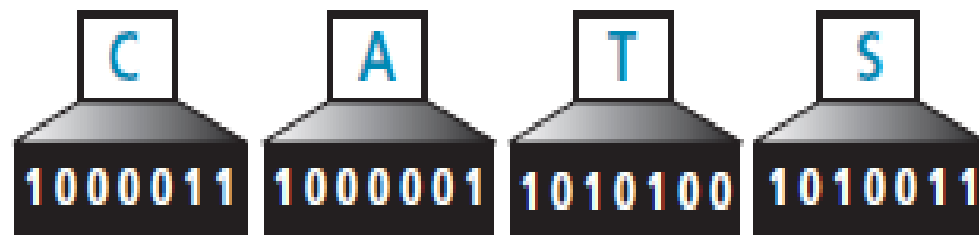


Table 3.3 Number of symbols and bit pattern length

<i>Number of symbols</i>	<i>Bit pattern length</i>	<i>Number of symbols</i>	<i>Bit pattern length</i>
2	1	128	7
4	2	256	8
8	3	65 536	16
16	4	4 294 967 296	32

- **ASCII** and **Unicode** are two most popular coding systems

ASCII Code (7-bit)

- The ANSI (American National Standard Institute) represented ASCII (American Standard Code for Information Interchange) character set.
- ASCII is a character encoding standard that uses 7 bits to represent 128 characters, including letters, numbers, punctuation marks, and control characters. It was originally designed for teleprinters and early computers and has become a widely used standard for encoding text.

0 – 31	32 – 127		
Control codes	Printable codes		
	48 –57	65 –90	97 – 122
	0, 1, 2, ... , 9	A, B, C, ... , Z	a, b, c, ... , z

ASCII Table from https://computersciencewiki.org/index.php?title=File:Ascii_table.png

[illegible]

Extended ASCII (8-bit)

- The original ASCII character set was a 7-bit encoding, 128 characters, embedded in an 8-bit byte, with the last bit (bit 7, value 128) reserved for “parity”, a simple check for transmission errors.
- As transmission systems became more reliable the parity check became redundant, and the final bit became available for extending the code table. Those additional 128 codes are the Extended ASCII character set (EASCII for short).
- At around the same time computing was becoming more widespread, and there was increasing interest in providing characters for non-English text. The EASCII range was taken up for accented(重音) letters, non-Latin alphabets and other special characters. The 128 code points could not cater for all these needs, so a whole range of language- and region-specific EASCII encodings was developed, documented in the [ISO-8859 family of standards](#).

Unicode

- **Unicode/ISO 10646** is an information technology standard for the consistent encoding, representation, and handling of text expressed in most of the world's writing systems - [a single encoding for the whole world](#).
- The standard, which is maintained by the Unicode Consortium, defines 144,697 characters covering 159 modern and historic scripts, as well as symbols, emoji, and non-visual control and formatting codes.
- Unicode can be implemented by different character encodings. The Unicode standard defines three Unicode Transformation Formats (UTF), UTF-8, UTF-16 and UTF-32, all in practice variable-length encodings, and several other encodings exist.

Unicode: A Table of Characters

- Original ISO/IEC design (32-bit): 128 groups of 256 planes
- Original Unicode design (16-bit): 16 bits per character, 2¹⁶ characters
- Final structure: 1+16=17 planes:
 - BMP (base multilingual plane, plane 0, modern-use)
 - Plane 1, SMP (supplementary multilingual plane, historic scripts,...)
 - Plane 2, SIP (Supplementary Ideographic Plane, really rare ideographs)
 - Planes 3-13: currently unused
 - Plane 14, SSP (Supplementary Special-purpose Plane, tags, variant selectors,...)
 - Planes 15 and 16: Private Use

UTF-8

- Multibyte encoding with nice properties
- ASCII-compatible in a very strong sense

UTF-8 Structure

from	to	usage	byte number			
			1	2	3	4
U+0000	U+007F	US-ASCII	0xxx xxxx	-	-	-
U+0080	U+07FF	Latin,..., Arabic	110x xxxx	10xx xxxx	-	-
U+0800	U+FFFF	rest of BMP	1110 xxxx	10xx xxxx	10xx xxxx	-
U+10000	U+10FFFF	non-BMP	1111 0xxx	10xx xxxx	10xx xxxx	10xx xxxx

UTF-16

- Code unit is 16 bits (two bytes, one half-word)
- One code unit for characters in BMP
- Two code units for characters in Planes 1-16
- 2048 "code points" in BMP reserved for UTF-16
- High surrogates: D800-DBFF
- Low surrogates: DC00-DFFF

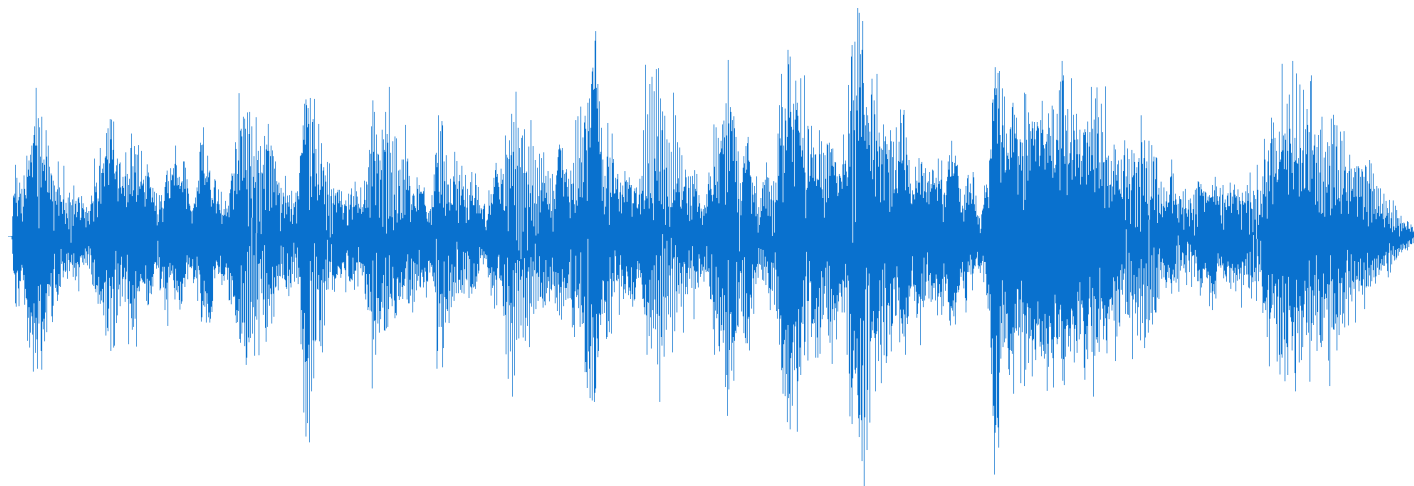
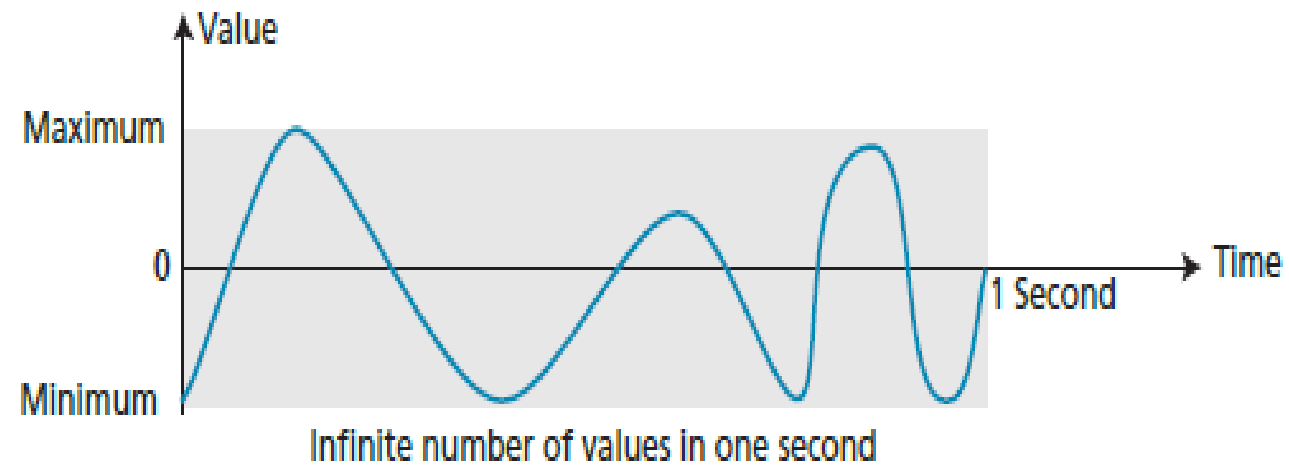
UTF-32

- Each character is directly represented as a 4-byte (32bits) code unit (one word)
- Straightforward, but rather inefficient encoding
- Used internally on some Unix-like systems and applications, and for certain kinds of processing
- Endianness problems and solutions are similar to those of UTF-16

3.4 STORING AUDIO

- **Audio is a representation of sound or music.** Audio, by nature, is different than the numbers or text we have discussed so far.
- Text is composed of countable entities (characters): we can count the number of characters in text. **Text is an example of digital data.**
- In contrast, audio is not countable. **Audio is an example of analog data.** Even if we are able to measure all its values in a period of time, we cannot store these in the computer's memory, as we would need infinite number of memory locations.
- Figure 3.15 shows the nature of an analog signal, such as audio, that varies with time.

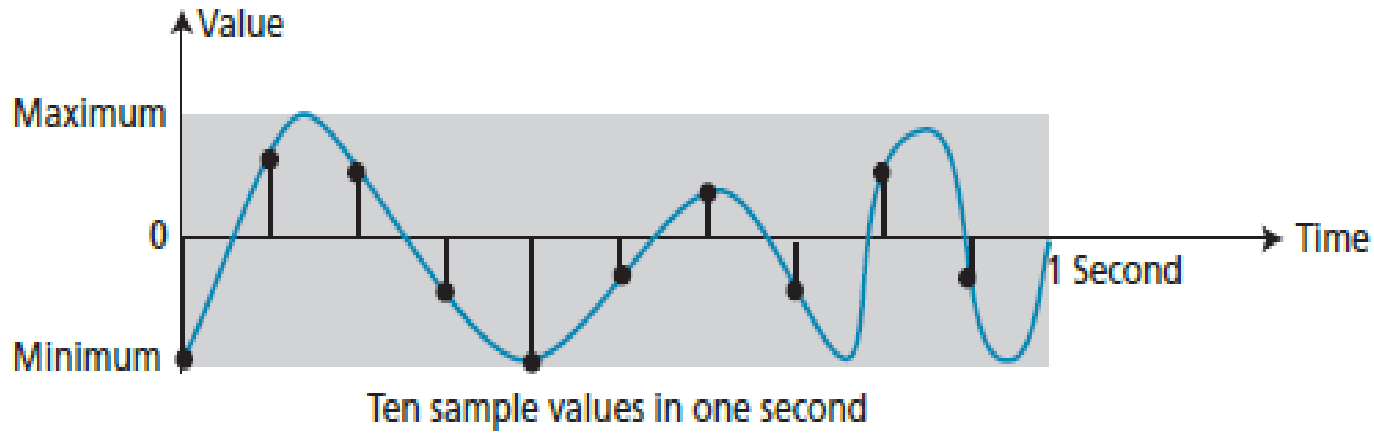
Figure 3.15 *An audio signal*



3.4.1 Sampling

- If we cannot record all the values of an audio signal over an interval, we can record some of them.
- Sampling means that we select only a finite number of points on the analog signal, measure their values, and record them.

Figure 3.16 *Sampling an audio signal*



3.4.2 Quantization

- The value measured for each sample is a real number. This means that we can store 40,000 real values for each one second sample.
- However, it is simpler to use an unsigned integer (a bit pattern) for each sample.
- Quantization refers to a process that rounds the value of a sample to the closest integer value.
- For example, if the real value is 17.2, it can be rounded down to 17: if the value is 17.7, it can be rounded up to 18.

3.4.3 Encoding

- The quantized sample values need to be encoded as bit patterns. Some systems assign positive and negative values to samples, some just shift the curve to the positive part and assign only positive values.
- If we call the **bit depth** or number of bits per sample B, the number of samples per second, S, we need to store $S \times B$ bits for each second of audio.
- This product is sometimes refer to as **bit rate**, R. For example, if we use 40,000 samples per second and 16 bits per each sample, the bit rate is

$$R = 40,000 \times 16 = 640,000 \text{ bits per second}$$

3.4.4 Standards for sound encoding

- Uncompressed audio formats, such as WAV, AIFF, AU, PCM.
- Audio formats with lossy compression – MP3, AAC, WMA
- Today the dominant standard for storing audio is **MP3** (short for **MPEG Layer 3**).
 - This standard is a modification of the **MPEG (Motion Picture Experts Group)** compression method used for video. It uses 44100 samples per second and 16 bits per sample.
 - The result is a signal with a bit rate of 705,600 bits per second, which is compressed using a compression method that discards information that cannot be detected by the human ear.

3.5 STORING IMAGES

Images are stored in computers using two different techniques: **raster graphics** and **vector graphics**.

3.5.1 Raster graphics

- **Raster graphics** (or **bitmap graphics**) is used when we need to store an analog image such as a photograph.
- A photograph consists of analog data, similarly to audio information: the difference is that the intensity (color) of data varies in space instead of in time.
- This means that data must be sampled. However, sampling in this case is normally called **scanning**.
- The samples are called **pixels** (picture elements).

Resolution

- Just like audio sampling, in image scanning we need to decide how many pixels we need to record for each square or linear inch. The scanning rate in image processing is called **resolution**. If the resolution is sufficiently high, the human eye cannot recognize the discontinuity in reproduced images.
- Dots per inch (DPI, or dpi) is a measure of spatial printing, video or image scanner dot density, in particular the number of individual dots that can be placed in a line within the span of 1 inch (2.54 cm).

Color depth

- The number of bits used to represent a pixel, its **color depth**, depends on how the pixel's color is handled by different encoding techniques.
- The perception of color is how our eyes respond to a beam of light. Our eyes have different types of photoreceptor cells: some respond to the three primary colors red, green, and blue (often called **RGB**), while others merely respond to the intensity of light.

True-Color

True-color uses **24** bits (8 bits for each of Red, Green, and Blue) to encode a pixel.

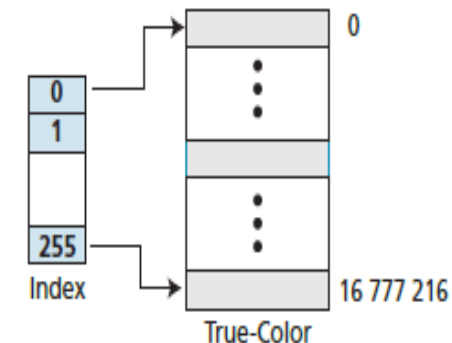
Table 3.4 Some colors defined in True-Color

Color	Red	Green	Blue	Color	Red	Green	Blue
Black	0	0	0	Yellow	255	255	0
Red	255	0	0	Cyan	0	255	255
Green	0	255	0	Magenta	255	0	255
Blue	0	0	255	White	255	255	255

Indexed color

Indexed color or **palette color** scheme uses only a portion of these colors.

Figure 3.17 Relationship of the indexed color scheme to the True-Color scheme



- For example, most digital cameras support images with 4K resolution (3840×2160 pixels).
- The following shows the number of bits that need to be stored using each scheme:

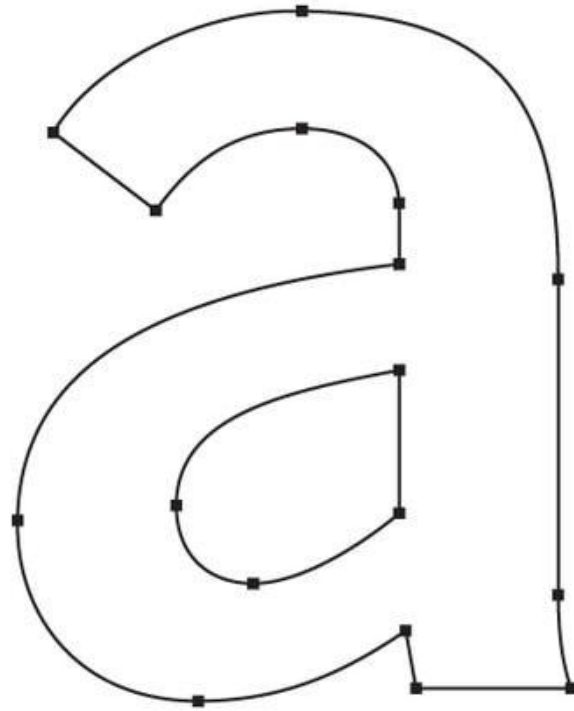
True-color	Index-color
$3840 \times 2160 \times 24 =$ 199,065,600 bits (199M)	$3840 \times 2160 \times 8$ = 66,355,200 bits (66M)



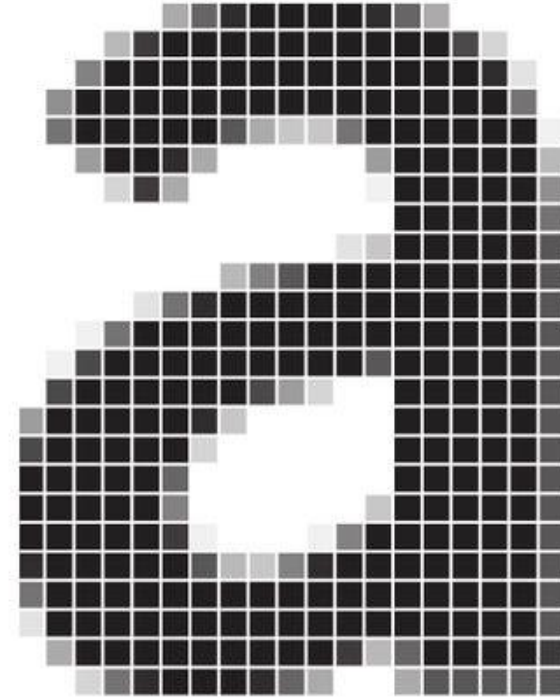
3.5.2 Vector graphics

- **Raster graphics** has two disadvantages:
 - the file size is big and rescaling is troublesome.
 - To enlarge a raster graphics image means enlarging the pixels, so the image looks ragged when it is enlarged.
- The **vector graphic** image encoding method, however, does not store the bit patterns for each pixel.
 - An image is decomposed into a combination of geometrical shapes such as lines, squares, or circles.
 - For example, consider a circle of radius r . The main pieces of information a program needs to draw this circle are:
 1. The radius r and equation of a circle.
 2. The location of the center point of the circle.
 3. The stroke line style and color.
 4. The fill style and color.





VECTOR



RASTER

Lossless Image File Formats

- **BMP (.bmp)** (Windows bitmap) handles graphic files within the Microsoft Windows OS.
- **RAW Image Files (.raw, .cr2, .nef, .orf, .sr2, and more)** RAW images are images that are unprocessed that have been created by a camera or scanner. Many digital SLR cameras can shoot in RAW.
- **TIFF (.tif, .tiff)** allowing for very high-quality images but also larger file sizes. (There are options for compression).
- **GIF (.gif)** uses the indexed color scheme.
- **PNG (.png)** Portable Network Graphics files are originally designed to improve upon and replace the gif format. PNG files are able to handle up to 16 million colors, unlike the 256 colors supported by GIF.
- **EPS (.eps)** An EPS or Encapsulated PostScript file is a common [vector file](#) type. EPS files can be opened in many illustration applications such as Adobe Illustrator or CorelDRAW.

Lossy Image File Formats

- **JPEG (.jpg, .jpeg)** Nearly every digital camera can save images in the JPEG format, which supports eight-bit grayscale images and 24-bit color images. JPEG applies lossy compression to images, which can result in a significant reduction of the file size. Applications can determine the degree of compression to apply, and the amount of compression affects the visual quality of the result. (JPEG also provides lossless image storage, but the lossless version is not widely supported.)
- **HEIF** - The High Efficiency Image File Format (HEIF) is an image container format that was standardized by MPEG on the basis of the ISO base media file format. While HEIF can be used with any image compression format, the HEIF standard specifies the storage of HEVC intra-coded images and HEVC-coded image sequences taking advantage of inter-picture prediction.

3.6 STORING VIDEO

- Video is a representation of images (called frames) over time.
- A movie consists of a series of frames shown one after another. In other words, video is the representation of information that changes in space and in time.
- So, if we know how to store an image inside a computer, we also know how to store video: each image or frame is transformed into a set of bit patterns and stored.
- The combination of the images then represents the video.
- Uncompressed video requires a very high data rate.
- Although lossless video compression codecs perform at a compression factor of 5 to 12, a typical H.264 lossy compression video has a compression factor between 20 and 200.

Common Video File Formats

- **MP4** is probably the most commonly used video file format for more than a decade now. It was developed by Apple and gained popularity with the iTunes player. MP4 containers can use H.264, H.265, and MPEG-4 codecs.
- **MOV** is another video file format that was developed by Apple Inc. It was the format used by Apple's Quicktime video player
- **AVI** stands for Audio Video Interleave. It is one of the oldest video file formats that is still popularly used. It was developed by Microsoft in 1992.
- **FLV** video format, popularly known as FLV, is another very commonly used video format. It was developed by Adobe Systems.
- **MKV** is an open-source video format. It was developed by Matroska, which is a non-profit organization based in France. Being open-source makes this video format common and popular in emerging content creators.
- **WMV** is short for Windows Media Video. WMV is both a codec format and also a video file container. WMV was developed by Microsoft as part of the Windows Media Framework.

Exercises

- 由Google搜尋HexEdit應用程式，練習查看與編輯各類文件檔案內容
 - MiteC HexEdit <https://www.mitec.cz/hex.html>
 - Web online HexEdit <https://hexed.it/>
 - PSPad <https://www.pspad.com/en/download.htm> 檢視[V] 選單→HEX編輯模式[X]
- P. 68 3.8.3 Problems
 - P. 3-8, P3-9, P3-14, P3-18, P3-20, P3-22, P3-26, P3-27, P3-28, P3-29, P3-30

Extend Readings

- [Bitmap vs. Vector Images: what's the difference?](#)
- [Video vs. Animation: what's the difference?](#)
- [Difference between WAV vs. MIDI file format](#)